SHS-HTTPS Enforcer: Enforcing HTTPS and preventing MITM Attacks

Sugavanesh B
National Institute of Technology, Tiruchirappalli, India
Department of Computer Science and Engineering
sugavaneshb@gmail.com

Hari Prasath R
National Institute of Technology, Tiruchirappalli, India
Department of Computer Science and Engineering
hariprar@gmail.com

Selvakumar S
National Institute of Technology, Tiruchirappalli, India
Department of Computer Science and Engineering
ssk@nitt.edu

ABSTRACT

Web servers provide immunity against Man In The Middle (MITM) attacks and eavesdropping by using HTTP Strict Transport Security (HSTS) to force user agents to communicate only over HTTPS connections. However, the initial connection request from a user is made over an insecure HTTP connection. This issue was addressed by user agents; Google Chrome and Firefox, implicitly, by including a static list of URLs to be accessed only over secure HTTPS connections. Since, these user agents maintain their lists independently, the URLs used by one user agent are invisible to another. A user is prone to MITM attacks, especially in public hotspot environments, when accessing a URL present in the list of secure URLs of one browser but not in another, since the initial handshake from that user agent is insecure. Attacks can be initiated by modifying the outgoing HTTP packets and also the HTTPS response packets from the webserver. This motivated us to propose a solution independent of user agents, by merging the static URL lists of different user agents and enforcing HTTPS for all those URLs. In this paper, we propose a solution, SHS-HTTPS Enforcer that introduces a local daemon to enforce URL redirection before the request flows out of the client for the URLs in a list compiled from multiple sources. The proposed solution has been demonstrated through a prototype implementation of the Squid Proxy server as our local daemon. The experiment was conducted by providing a URL, which was not present in one browser's list but was present in another browser's list. It was evident that SHS-HTTPS Enforcer enforced HTTPS successfully and MITM attacks were prevented.

Categories and Subject Descriptors

C.2.0 [**Computer-Communication Networks**]: General – *security and protection.*

General Terms

Design, Experimentation, Security, Human Factors, Theory, Verification

Keywords

HTTPS deployment, HSTS, man-in-the-middle attack, browser security, URL redirection

1. INTRODUCTION

HTTPS was designed by layering HTTP [1] over Transport Layer Security (TLS) [2] to provide end-to-end protection against eavesdropping, man in the middle attacks (MITM) [3] and other network attacks. In response to increasing attacks against online transaction portals, email service providers, social networking sites, and websites containing sensitive information, HTTPS was made the default option for communication between client and server by encrypting all their contents. However, a man in the middle attack can be initiated by stripping the enforce HTTPS response packet using the SSLStrip attack [4]. Since the browser has no information regarding whether the website has to be accessed in a secure fashion, such attacks go unwarned and unnoticed by users. This can lead to leaking of sensitive information. Also through active MITM attacks using ARP poisoning or DNS poisoning, users can be redirected to bogus websites posing as original websites by providing self-signed certificates to user agents. During such events the user agent generally issues a warning regarding the validity of the certificates. Studies show that more than 50% of users [5] or more than 80% of users [6] bypassed such warnings.

MITM attacks can be prevented if the user agents use HTTPS. To inform the user agent that a connection to the web server should always use HTTPS, the HTTP Strict Transport Security (HSTS) [7] was proposed.

HSTS is supported by the following browsers, Google Chrome (4.0.211.0 onwards), Firefox (4 onwards), and Opera (12 onwards) only. It is not known if other user agents will be implementing HSTS in the near future.

HSTS employs the trust-on-first use model used by SSH. This leads to the possibility of a MITM attack, known as SSLStrip [4], during the initial connection stage by stripping the enforce HTTPS response packet. SSLStrip tools can be easily obtained from the internet, which increases the possibilities of an attack, thus nullifying the effect of HSTS. Google Chrome and Firefox came up with a list of secure URLs that need to be accessed through HTTPS as insisted by corresponding web servers.

2. EXISTING SOLUTIONS

2.1 SSLock

SSLock [8] is a universal SSL enforcement mechanism and it employs the domain segmentation approach. The web server attaches an extra HTTP header, *SSLock-Candidates* (indicating the subdomains to be accessed in a secure manner). The user agent at the client side employs redirection-scripts to read *SSLock-Candidates* and redirects to a secure domain accordingly. Thus all *SSLock-Candidates* domains are immune to SSL Stripping. However, this method is limited to Firefox since it relies on the library of Firefox.

2.2 HTTPSLock

HTTPSLock [9] is a stricter solution than other solutions to enforce valid SSL certificates. All secure connections with invalid certificates are automatically refused on behalf of users by employing strict certificate verification authorities. If an invalid SSL certificate is detected, an error-message is displayed. This prevents SSL sniffing attacks that use fake SSL certificates. Also, if the web address that needs to be accessed only using HTTPS is not accessed using HTTPS, then it is redirected to an error-message page. At present, 72% of the browsers support HTTPSLock [9]. It detects the possibility of an attack but does not provide any ways to overcome it. Such measures may lead to Denial of Service attacks on sites where HTTPS is incorrectly implemented by webmasters or on sites that use self-signed certificates.

DOI: 10.1145/2532780.2532802 http://doi.acm.org/10.1145/2532780.2532802

2.3 ForceHTTPS

ForceHTTPS [10] uses cookies to guide the browser to be more secure. By setting a ForceHTTPS cookie, the browser treats HTTPS errors as attacks and not as simple configuration mistakes. The behavior of the browser is modified as follows when ForceHTTPS is enabled:

i) Any Non-HTTPS connection to the cookie set site is redirected to HTTPS
ii) A TLS session is terminated when any TLS errors occur, including self-signed certificates and common-name mismatches.
iii) Any attempt to embed insecure (non-HTTPS) content such as SWF movies or cascading style sheets leads to a warning about possible network attacks. By enabling ForceHTTPS, a site protects itself from the mistakes of its own web developers.

Since ForceHTTPS uses cookies as a means to store the information about HTTPS redirection, the information is lost when the cookies and temporary files are deleted. It also employs the trust-on-first-use model, which is vulnerable to MITM attacks.

2.4 HSTS

HSTS [7] (based on ForceHTTPS and HTTPSLock) is a web security policy mechanism where the web server forces a client to use HTTPS connections to access its web content. The web server attaches a special header called an HSTS header in the response packet, which intimates the user agent the following:

i) Request should be made through HTTPS
ii) Sub-domains to be accessed following this rule
iii) Max age of the rule (time period for the user agent to remember this enforcement)

HSTS includes the hard-coded domain names of *must-be-https* URLs in a list, which is maintained by the respective browsers. It prevents MITM attacks such as SSLStrip, but it is limited to the prelisted URLs. At present, Chrome and Firefox are the browsers that have implemented HSTS at a primitive level and maintain separate lists of *must-be-https* URLs.

2.5 ISAN-HTTPS Enforcer

ISAN-HTTPS Enforcer [11] makes use of the Javascript API to enforce redirection to https. When the web server responds to a request from the user agent, it also responds with messages and scripts to indicate redirection to HTTPS. The Enforcer at the client side checks the list of URLs and redirects to HTTPS accordingly. This is supported by all browsers that support JavaScript. Webmasters can easily develop their web pages to enforce HTTPS by just including and calling the JavaScript API with no need for modification or loading of any extensions into the user agents in the client side. An active MITM attack is still possible by stripping the call to JavaScript API in the response packet.

3. PROPOSED SOLUTION

3.1 Design of SHS-HTTPS Enforcer

In this paper, we propose a solution, SHS-HTTPS Enforcer, with no user interaction that is built over the existing solutions. The static HSTS list maintained by Chrome and Firefox are independent of each other and they may complement each other in most cases. This leads to a situation wherein the user becomes vulnerable to attack if the user changes the user agent for accessing a website which is present in the list of *must-be-https* URLs of other user agent. Neither is a common list maintained across all user agents nor is the original HSTS made part of the security architecture of all user agents. SHS-HTTPS Enforcer circumvents both situations by making use of the lists from existing sources [14], [15] and enforcing HTTPS at the system level rather than at the browser level as in the existing solutions [8], [9], [10], [11].

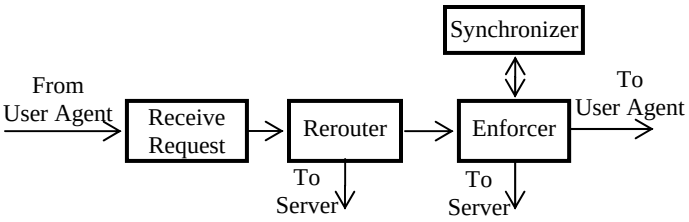

Figure 1: Block Schematics of SHS-HTTPS Enforcer

Figure 2: Rerouter Module – Rerouting HTTP traffic

The details of SHS-HTTPS Enforcer are as follows:

1) When users enter the URL in the address bar of the user agent, they generally tend to type without mentioning *http* or *https*. In most cases the initial webpage request is made through *http*. SHS-HTTPS Enforcer monitors all outgoing traffic and redirects all *http* traffic to a local daemon.
2) In the local daemon, the URL is stripped from incoming packets and is checked against the preloaded list. If a match for the URL is found, then it is redirected to *https*. Henceforth, the communication between the client and webserver uses *https* while rest of the traffic remains unaffected.
3) The preloaded list is periodically synced with the static list maintained by the Chrome and Firefox developers.

4. IMPLEMENTATION OF SHS-HTTPS ENFORCER

The implementation of SHS-HTTPS Enforcer is given in the block schematic shown in Figure 1. The individual setup and interaction steps are discussed in the following Sections:

4.1 Rerouter Module – Rerouting HTTP traffic

Squid [12] is a proxy server and web cache daemon. It is a lightweight server and has low overhead. SHS-HTTPS Enforcer makes use of the Squid server as a local daemon. When the browser establishes a connection to the server, the initial HTTP request is sent via port 80. Using iptables [13], a rule is written to redirect all outgoing traffic from port 80 to our Squid server listening to a different port, say 3128 as shown in Figure 2.

4.2 Enforcer Module – Conversion of HTTP to HTTPS

The URL is stripped from the request packet received by the Squid server. Using regular expressions the URL is compared against the preloaded URLs to verify whether the request should be converted to an HTTPS request packet, as shown in Figure 3. If the URL is to be converted, an HTTP response status code 302 is returned and redirection to HTTPS is enforced. The exchange of HSTS parameters and the public key handshake will be handled by the browser.

4.2.1 Algorithm for conversion of HTTP to HTTPS

The algorithm for the Enforcer module, which converts HTTP to HTTPS is as follows:

1. Get RequestURL
2. Compare with STS list
3. If Match
    a. Assemble response packet with status code (302)
    b. Enforce redirection to HTTPS
4. If NO Match
    a. Direct RequestURL to interface

DOI: 10.1145/2532780.2532802 http://doi.acm.org/10.1145/2532780.2532802

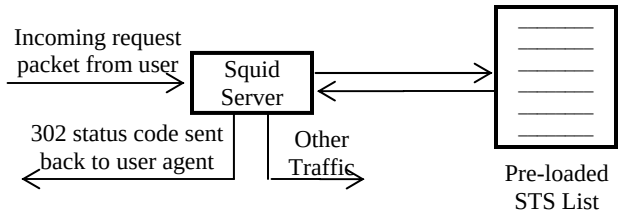

Figure 3: Enforcer Module – Conversion of HTTP to HTTPS

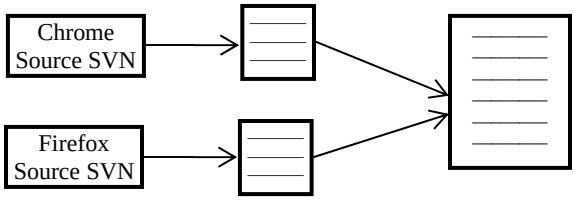

Figure 4: Synchronizer Module – Syncing STS Lists

4.3 Synchronizer Module – Syncing STS list

The list of URLs in the HSTS preloaded list was fetched from Chrome [14] and Mozilla [15] source repositories. Both the lists are synced against the list maintained in the Squid server and it is used to check against the live incoming URLs, as shown in Figure 4. This module runs periodically to reflect the changes in the sources [14], [15].

Table 1: Summary of implementation details of SHS-HTTPS Enforcer

Sl. No.	Module	Function	Calls	Called By	Remarks
1	Rerouter	-	-	-	iptable rule was used
2	Enforcer	compare()	rewrite()	Squid Server	Uses output of synchronizer
		rewrite()	-	compare()	Status code 302 is embedded
3	Synchro-nizer	chrome_get()	-	Squid Server	Chrome list obtained
		firefox_get()	-	Squid Server	Firefox list obtained
		merge()	-	Squid Server	The lists from Firefox and Chrome are merged

4.4 Implementation Summary

The implementation summary of SHS-HTTPS Enforcer is given in Table 1.

Figure 5.a: The URL http://example.com was entered in the address bar of user agent Firefox

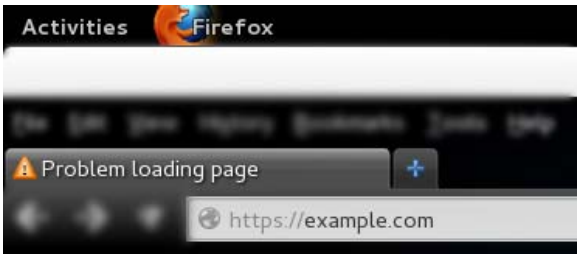

Figure 5.d: Redirection was successful and outgoing request is through HTTPS

```
Starting Squid Cache version 3.3.4 for x86_64-unknown-linux-gnu...
Process ID 16688
Process Roles: master worker
With 1024 file descriptors available
Initializing IP Cache...
DNS Socket created at [::], FD 5
DNS Socket created at 0.0.0.0, FD 6
Adding nameserver 192.168.43.1 from /etc/resolv.conf
helperOpenServers: Starting 0/20 'rewrite.pl' processes
helperOpenServers: No 'rewrite.pl' processes needed.
Logfile: opening log /var/log/squid/access.log
```

Figure 5.b: Squid server running as local daemon

```
1368623467.832      4 ::1 TCP_REDIRECT/302 261 GET http://example.com/ - HIER_NONE/
1368623468.365    147 ::1 TCP_MISS/503 0 CONNECT example.com:443 - HIER_NONE/- -
1368623492.027      3 ::1 TCP_REDIRECT/302 261 GET http://example.com/ - HIER_NONE/
1368623492.168      3 ::1 TCP_MISS/503 0 CONNECT example.com:443 - HIER_NONE/- -
```

Figure 5.c: Squid access log entry showing redirection to HTTPS (via port 443)

DOI: 10.1145/2532780.2532802 http://doi.acm.org/10.1145/2532780.2532802

5. EXPERIMENT

An *iptable* rule was added to redirect http traffic to the Squid server. The Squid server was started at port 3128, and was configured to run a Perl Script to redirect URLs. The Perl script performed URL stripping and checking against the URLs from the preloaded list. A *cron* job initiated another Perl script to periodically update the preloaded list.

To automate the entire process, a shell script that performs the functions of adding the iptable rule, starting the Squid server with a predefined configuration and *cron* job was initiated at the boot.
For the experiment, the URL http://example.com was added to the STS list manually. It is evident that the URL http://example.com does not have an HTTPS provision. This implies that there will be no HTTPS enforcement header in the response packet from the corresponding server. Since the Squid server was configured to run the modules of SHS-HTTPS Enforcer, http://example.com was redirected to https://example.com happened. HTTPS traffic uses port 443 for communication by default [2]. The redirection to HTTPS was also evident from the Squid server's access log entry showing redirection to HTTPS via port 443, as shown in Figure 5.c. The sequence of steps leading to successful redirection of URL, http://example.com, is depicted in Figure 5.

The SHS-HTTPS Enforcer was tested by creating a hostile environment and launching a MITM attack. The experiments were carried out in machines with Intel® Core™ i5 2.40 GHz processor, 4GB RAM, and Arch Linux 3 as Operating System. The Python tool SSLStrip [16] was used to launch the attack. The experimental results and performance evaluation are as follows:

5.1 Results and Performance Evaluation

Browser Support: SHS-HTTPS Enforcer was built as a system-wide security solution independent of the browser. So, it works irrespective of the browser. It also supports user agents, curl and lynx.

User friendliness: The initial setup was made by running a shell script. It was evident from the experiments conducted that there was no user interaction.

Effectiveness: Around 85 URLs were listed in Mozilla Source SVN [15], as of May 15th, 2013 while SHS-HTTPS Enforcer maintains a list of 472 URLs. So, SHS-HTTPS Enforcer has made Firefox more secure against MITM attacks. Also, irrespective of the user agents used, the redirection was enforced for all URLs. The effectiveness in detecting and protecting various forms of attacks in comparison to other existing solutions are summarized in Table 1.

From Table 1, it is clearly seen that SHS-HTTPS Enforcer is more effective against attacks by overcoming the limitations of other solutions while incorporating the positive aspects of other solutions.

6. CONCLUSIONS

In this paper, the possibility of MITM attacks in HSTS due to the vulnerability of insecure initial handshakes is addressed. The proposed solution, SHS-HTTPS Enforcer, imposes HTTPS during the initial handshake by URL redirection before the request flows out. The proposed SHS-HTTPS Enforcer was tested by performing MITM attacks using the SSL stripping technique. Common performance measures such as Browser independency, User friendliness, and Effectiveness of the security were used for evaluation of our method in comparison to other solutions and the working of our solution was demonstrated through experimental results. From the experiments conducted it was evident that the SHS-HTTPS Enforcer enforced HTTPS successfully.
The SHS-HTTPS Enforcer can be overridden when a user uses a proxy to connect to internet, since the packets will be addressed to the port number defined by the proxy. Our ongoing research is to enforce redirection to HTTPS at the client's firewall/interface and to implement it as an operating system feature.

7. REFERENCES

[1] R. Fielding, J. Gettys, J. Mogul, Henrik Frystyk, L. Masinter, P. Leach, T. Berners-Lee, "RFC 2616: Hypertext Transfer Protocol - HTTP/1.1", IETF, RFC 2616, June 1999.

[2] E. Rescorla, "HTTP Over TLS", IETF, RFC 2818, May 2000.

[3] M. Howard, "Man-in-the-Middle Attack to the HTTPS Protocol", IEEE computer society, pp.78-81, February 2009.

[4] M. Marlingspike, "New Tricks For Defeating SSL in Practice", BlackHat Conference, USA, 2009.

[5] S.E. Schechter, R. Dhamija, A. Ozment, I. Fischer, "The Emperor's New Security Indicators", in Proceedings of the 2007 IEEE Symposium on Security and Privacy, pp.51-65, May 20-23, 2007.

[6] J.Sunshine, S. Egelman, H. Almuhimedi, N. Atri, L.F. Cranor, "Crying Wolf: An Empirical Study of SSL Warning Effectiveness", in Usenix Security Symposium, August 10-14, 2009, Montréal, Canada.

[7] J. Hodges, C. Jackson, A. Barth, "HTTP Strict Transport Security (HSTS)", IETF, RFC 6797, Internet draft, June 2012.

[8] A. Fung and K Cheung, "SSLock: Sustaining the Trust on Entities Brought by SSL", In Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security (ASIACCS10), Beijing, China, April 2010.

[9] Adonis P.H. Fung, K.W. Cheung, "HTTPSLock: Enforcing HTTPS in unmodified browsers with cached JavaScript", 2010 Fourth International IEEE Conference on Network and System Security, IEEE, pp. 269-274, September 2010.

[10] Collin Jackson, Adam Barth, "ForceHTTPS: Protecting High-Security Web Sites from Network Attacks", in 17th International World Wide Web Conference Committee, Beijing, China, April 2008. https://crypto.stanford.edu/forcehttps

[11] Somnuk Puangpronpitag, Nattavut Sriwiboon "Simple and Lightweight HTTPS Enforcement to Protect Against SSL Striping Attack", 2012 Fourth International Conference on Computational Intelligence, Communication Systems and Networks, pp. 229-234, July 2012.

[12] Squid Proxy Server. http://www.squid-cache.org [Accessed on May 15th, 2013]

[13] IP Tables. http://www.netfilter.org/projects/iptables/ [Accessed on May 15th, 2013]

[14] Chrome HSTS static list. http://src.chromium.org/viewvc/chrome/trunk/src/net/http/transport_security_state_static.json [Accessed on May 15th, 2013]

[15] Mozilla HSTS static list https://mxr.mozilla.org/mozillacentral/source/security/manager/boot/src/nsSTSPreloadList.inc [Accessed on May 15th, 2013]

[16] SSLStrip, a Python tool to hijack HTTPS connections during an MITM attack. http://www.thoughtcrime.org/software/sslstrip/ [Accessed on May 15th, 2013]